

[보컬녹음이 가능한 미디시퀀서 애플리케이션]

[MIDI Sequencer Application With Vocal Recording]

권택준

2019 년 11 월 27 일

요 약

MIDI(Musical Instrument Digital Interface)는 전자 악기 간 통신규약으로, 연주 정보를 담고 있어 저용량으로 음을 표현할 수 있다는 장점이 있지만 사람의 목소리와 같은 실제 소리를 표현할 수 없는 한계점이 있습니다. 이것을 해결해보기 위해 먼저 Android Studio 개발 환경에서 MIDI Sequencer UI를 제작하여 사용자가 MIDI를 작곡할 수 있는 환경을 만들고, 작곡된 MIDI 파일(.mid)에 보컬을 녹음해서 하나의 Waveform Audio 파일(.wav)로 결합시킵니다. 본 문서는 위의 기술에 대한 원리를 설명합니다.

목 차

I .	서 론	4
1.1	MIDI의 역사	4
1.2	MIDI의 한계	4
II .	사용 이론 및 기술	5
2.1	MIDI File Format	5
2.2	MIDI Sequencer UI	9
2.3	WAV File Format	11
III .	작품 구현 및 실험결과	13
IV .	결론	16
V .	참고문헌	16

표 및 그림 목 차

[표 2-1]		6
[표 2-2]		7
[표 2-3]		7
[표 2-4]		8
[그림 2-1]		9
[그림 2-2]		10
[그림 2-3]		11
[그림 3-1]		13
[그림 3-2]		14
[그림 3-3]		14
[그림 3-4]		15
[그림 3-5]		15
[그림 3-6]		15
[그림 3-7]		16

I . 서론

1. MIDI의 역사

1960년대 로버트 모그가 현대적인 신디사이저의 상업화에 성공하게 되면서 다양한 전자악기들이 등장하였으며 1970년대 말부터는 폭발적으로 증가하기 시작했습니다. 전자악기 제조사들은 저마다 독자적인 방식으로 악기 간의 통신을 위한 새로운 방법을 고안했는데, 이것이 회사마다 천차만별로 달랐기 때문에 다른 회사에서 만들어진 전자악기에는 무용지물이었으며 이 문제를 해결하기 위해 각종 신호 변환기가 잠시 등장하기도 했습니다. 표준화의 필요성을 느꼈던 각 전자악기 회사들은 데이비드 스미스와 쳇 우드가 고안하여 AES(Audio Engineering Society)에 공개한 인터페이스 표준 규격을 기반으로 회사 간 협의를 통해 수정해서 MIDI(Musical Instrument Digital Interface) 표준을 제정하게 되었습니다.

MIDI는 원래 전자악기를 사용하려는 전문 음악가와 레코드 제작자를 위한 것이었으나 개인용 컴퓨터(PC) 시대가 시작되면서 PC에 장착할 수 있는 MIDI 인터페이스가 등장하게 되었고 전문적인 연주 기술이 없는 사용자도 연주 데이터를 입력하는 것만으로 작곡과 편곡을 할 수 있게 되었습니다. 이로 인해 수많은 개인 창작물들이 쏟아져 나오게 되었고, 음악의 판도가 혁명적으로 바뀌었습니다.

2. MIDI의 한계

MIDI는 일반 음악 파일과는 달리 어떻게 소리를 만들어 내야 하는지에 대한 정보만을 담고 있으며 이 정보들을 샘플러(실제 소리를 디지털 방식으로 녹음해서 표본화해놓은 것을 임의로 재생 출력할 수 있는 장치)로 보내서 재생시키므로 저용량으로 음을 표현할 수 있다는 장점이 있지만 동시에 사람의 목소리와 같이 샘플러에 존재하지 않는 소리들은 표현할 수 없기 때문에 이것이 한계점이 되어버리기도 합니다. 20세기 대부분의 대중음악은 보컬(Vocal)과 반주의 혼합으로 구성되며, 뉴욕타임스가 지난 천 년의 어떤 악기도 따라올 수 없는 무한한 선율을 가진 최고의 악기를 인간의 목소리로 선정했을 정도로 현대 실용 음악에서는 보컬의 중요성이 강조되기 때문에 이 한계점은 더욱 두드러집니다. 따라서 누구나 쉽게 작곡할 수 있는 MIDI의 장점을 살리면서 보컬을 혼합시킬 수 있도록 하기 위해, Android Studio 개발환경에서 애플리케이션을 제작하여 사용자가 편리한 UI를 통해 MIDI 작곡을 하게 한 후, 샘플러에 보내져 재생되는 반주에 사용자의 목소리를 녹음하게 했습니다. 또한, 녹음되어 디지털 부호화(Pulse Code Modulation)된 소리를 마이크로소프트 윈도우의 표준 오디오 포맷인 WAV 파일로 가공했습니다.

II . 사용 이론 및 기술

1. MIDI File Format

MIDI 파일의 내용은 하나의 Header Chunk와 그 뒤에 붙는 하나 이상의 Track Chunk로 이루어집니다. 청크(Chunk)는 “덩어리”라는 뜻을 가지고 있으며 어떤 데이터의 집합을 일컫기 위해 자주 사용하는 표현입니다.

가. Header Chunk

Header Chunk는 Header Chunk임을 표시하는 아스키 코드(ASCII CODE) 문자열, Header Chunk의 데이터 길이, MIDI파일의 표현 형식, Track Chunk의 개수, 4분 음표의 delta time으로 구성됩니다.

1) Header Chunk임을 표시하는 아스키 코드 문자열(4바이트)

MIDI Track header를 줄인 “MThd”를 아스키 코드로 표현하며 각각 M(77), T(84), h(104), d(100) 입니다.

2) Header Chunk의 데이터 길이(4바이트)

이후에 올 MIDI파일의 표현 형식, Track Chunk의 총 개수, 4분 음표의 delta time의 길이입니다. 각각 2바이트씩 할당되어 총 길이는 6바이트가 되고 이를 4바이트로 표현해야 하므로 0, 0, 0, 6 입니다.

3) MIDI파일의 표현 형식(2바이트)

MIDI파일의 표현 형식은 총 3가지(0, 1, 2)가 있습니다. 0은 Header Chunk와 단일 Track Chunk로 구성되며 하나의 Track Chunk에 모든 정보가 포함되어 있어 컴퓨터가 연주하기 쉬운 형식입니다. 1은 Header Chunk와 복수의 Track Chunk로 구성되며 첫 번째 Track Chunk에 악보의 정보를 기록하고 이후의 Track Chunk에는 연주 정보를 담습니다. 모든 Track은 동시에 연주되면서도 따로 구분 지어져 있기 때문에 편집하기 쉬운 형식입니다. 2는 1과 유사하지만 모든 Track이 독립적으로 연주되며 거의 사용되지 않습니다. 제 애플리케이션에서 만들어지는 MIDI파일은 표현 형식1을 사용할 것이기 때문에 0, 1 입니다.

4) Track Chunk의 개수(2바이트)

Track Chunk의 개수는 사용자가 애플리케이션으로 작곡한 데이터 트랙의 개수에 따라 유동적으로 변하게 되며 2바이트(16bit)로 표현되어야 하므로 보유할 수 있는 트랙의 최대 개수는 $2^{16}-1$ 이 될 것입니다.

5) 4분 음표의 delta time(2바이트)

Track Chunk에서 음표의 길이는 4분 음표의 delta time을 기준으로 작성됩니다.

애플리케이션에서 사용할 음표는 셋 잇단 8분 음표, 셋 잇단 16분 음표, 8분 음표, 16분 음표, 32분 음표이며 각각의 delta time은 4분음표의 1/6, 1/12, 1/2, 1/4, 1/8로 표현될 수 있습니다. 따라서 4분 음표의 delta time을 24로 정해야 하므로 0, 24 입니다.

나. Track Chunk

MIDI파일의 표현 형식1에서는 복수의 Track Chunk가 사용되며 Track Chunk임을 표시하는 아스키 코드 문자열, Track Chunk의 데이터 길이, Track의 이벤트로 구성됩니다.

1) Track Chunk임을 표시하는 아스키 코드 문자열(4바이트)

MIDI Track을 줄인 “MTrk”를 아스키 코드로 표현하며 각각 M(77), T(84), r(114), k(107)입니다.

2) Track Chunk의 데이터 길이(4바이트)

첫 번째 Track Chunk인 경우에는 악보의 정보를 나타내는 Meta이벤트의 길이 + Track의 종료를 나타내는 Meta이벤트의 길이가, 두 번째 Track Chunk부터는 사용자가 추가하는 MIDI이벤트(전자음의 연주 정보)의 길이 + Track의 종료를 나타내는 Meta이벤트의 길이가 될 것이며 4바이트(32bit)로 표현되어야 하므로 Track Chunk 하나의 최대 데이터 길이는 $2^{32}-1$ 이 될 것입니다.

3) Track의 이벤트

Track 이벤트의 종류에는 Meta이벤트, MIDI이벤트, SysEx이벤트가 있습니다.

Meta이벤트는 MIDI 음악의 다양한 정보들을 나타냅니다.

[표 2-1]

형식(Byte)	역할
255 0 2 <시퀀스 번호(2바이트)>	트랙의 번호
255 1 <텍스트 길이(가변길이)> <텍스트>	트랙의 주석
255 2 <텍스트 길이(가변길이)> <텍스트>	저작권 고지
255 3 <텍스트 길이(가변길이)> <텍스트>	트랙 이름
255 4 <텍스트 길이(가변길이)> <텍스트>	트랙에서 사용된 악기에 대한 설명
255 5 <텍스트 길이(가변길이)> <텍스트>	음악의 가사
255 6 <텍스트 길이(가변길이)> <텍스트>	음악에서 어느 구간인지를 표시
255 7 <텍스트 길이(가변길이)> <텍스트>	무대에서 일어나는 일에 대한 설명
255 81 3 <BPM(3바이트)>	곡의 빠르기(BPM)을 설정
255 88 4 <분자> <분모(2^x)> <MIDI클럭> <클럭 당 32분음표 수>	곡의 박자를 표시
255 89 2 <기준음> <장·단조>	곡의 조표를 표시
255 47 0	트랙의 끝(필수)

MIDI 이벤트는 MIDI 음악 연주의 동작 및 제어 정보들을 나타냅니다.

[표 2-2]

형식(Byte)	역할
0 <192+[채널0-15]> <악기 번호>	해당 채널에 악기 지정
<delta time(가변길이)> <144+[채널0-15]> <음> <속도>	음 켜기(건반 누르기)
<delta time(가변길이)> <128+[채널0-15]> <음> <속도>	음 끄기(건반 떼기)

채널10(9)은 드럼이 약속된 채널이기 때문에 악기를 할당할 수 있는 채널은 총 15개(0-8, 10-15)이며, General MIDI로 등록된 악기 128개(0-127)를 채널10을 제외한 각 채널에 지정하여 사용할 수 있습니다.

[표 2-3]

General MIDI							
Piano		Chromatic Percussion		Organ		Guitar	
1	Acoustic Grand Piano	9	Celesta	17	Drawbar Organ	25	Acoustic Guitar(nylon)
2	Bright Acoustic Piano	10	Glockenspiel	18	Percussive Organ	26	Acoustic Guitar(steel)
3	Electric Grand Piano	11	Music Box	19	Rock Organ	27	Electric Guitar(jazz)
4	Honky-tonk Piano	12	Vibraphone	20	Church Organ	28	Electric Guitar(clean)
5	Electric Piano 1	13	Marimba	21	Reed Organ	29	Electric Guitar(muted)
6	Electric Piano 2	14	Xylophone	22	Accordion	30	Overdriven Guitar
7	Harpsichord	15	Tubular Bell	23	Harmonica	31	Distortion Guitar
8	Clavinet	16	Dulcimer	24	Tango Accordion	32	Guitar harmonics
Bass		Strings		Ensemble		Brass	
33	Acoustic Bass	41	Violin	49	String Ensemble 1	57	Trumpet
34	Electric Bass(finger)	42	Viola	50	String Ensemble 2	58	Trombone
35	Electric Bass(pick)	43	Cello	51	Synth Strings 1	59	Tuba
36	Fretless Bass	44	Double Bass	52	Synth Strings 2	60	Muted Trumpet
37	Slap Bass 1	45	Tremolo Strings	53	Voice Aahs	61	French horn
38	Slap Bass 2	46	Pizzicato Strings	54	Voice Oohs	62	Brass Section
39	Synth Bass 1	47	Orchestral Harp	55	Synth Voice	63	Synth Brass 1
40	Synth Bass 2	48	Timpani	56	Orchestra Hit	64	Synth Brass 2
Reed		Pipe		Synth Lead		Synth Pad	
65	Soprano Sax	73	Piccolo	81	Lead 1(square)	89	Pad 1(new age)
66	Alto Sax	74	Flute	82	Lead 2(sawtooth)	90	Pad 2(warm)
67	Tenor Sax	75	Recorder	83	Lead 3(calliope)	91	Pad 3(polysynth)
68	Baritone Sax	76	Pan Flute	84	Lead 4(chiff)	92	Pad 4(choir)
69	Oboe	77	Blown Bottle	85	Lead 5(charang)	93	Pad 5(bowed)
70	English Horn	78	Shakuhachi	86	Lead 6(voice)	94	Pad 6(metallic)
71	Bassoon	79	Whistle	87	Lead 7(fifths)	95	Pad 7(halo)
72	Clarinet	80	Ocarina	88	Lead 8(bass+lead)	96	Pad 8(sweep)
Synth Effects		Ethnic		Percussive		Sound Effects	
97	FX 1(rain)	105	Sitar	113	Tinkle Bell	121	Guitar Fret Noise
98	FX 2(soundtrack)	106	Banjo	114	Agogo	122	Breath Noise
99	FX 3(crystal)	107	Shamisen	115	Steel Drums	123	Seashore
100	FX 4(atmosphere)	108	Koto	116	Woodblock	124	Bird Tweet

101	FX 5(brightness)	109	Kalimba	117	Taiko Drum	125	Telephone Ring
102	FX 6(goblins)	110	Bagpipe	118	Melodic Tom	126	Helicopter
103	FX 7(echoes)	111	Fiddle	119	Synth Drum	127	Applause
104	FX 8(sci-fi)	112	Shanai	120	Reverse Cymbal	128	GunShot

악기 지정 이벤트를 통해 General MIDI 악기를 지정한 후에는 음 켜기/끄기 이벤트([표 2-2])를 활용하여 전자음을 연주시킬 수 있습니다. 해당 이벤트가 어느 시점에서 발생할 이벤트인지를 결정하는 역할을 하는 delta time은 Header Chunk에서 선언한 4분음표의 delta time을 기준으로 작성됩니다. 저는 Header Chunk에서 4분음표의 delta time을 24로 선언해놓았으므로 8분음표 길이의 delta time은 12가 될 것입니다. 그런데 좀 긴 길이의 음을 표현하기 위해 127이 넘는 delta time을 사용하려 한다면 그 때는 가변 길이(Variable Length, 최대 4바이트까지 허용)를 사용해야 합니다. 127이 넘어 캐리가 발생하게 되면 상위 바이트에 1을 더하게 되는데 가변길이에서 상위 바이트는 128부터 시작해 더해지며, 이것은 뒤의 값도 가변길이임을 알리기 위한 약속입니다. 예를 들어서 144의 delta time을 사용하고자 한다면 1번의 캐리가 발생하고 16이 남으므로, 가변길이는 129 16이 될 것입니다.

이벤트의 delta time을 정했다면 이벤트에서 어떤 음이 쓰여야 할지도 정해야 할

	드럼 채널 10	나머지 채널		드럼 채널 10	나머지 채널
35	Acoustic Bass Drum	B2	59	Ride Cymbal 2	B4
36	Bass Drum1	C3	60	Hi Bongo	C5
37	Side Stick	C#3	61	Low Bongo	C#5
38	Acoustic Snare	D3	62	Mute Hi Conga	D5
39	Hand Clap	D#3	63	Open Hi Conga	D#5
40	Electric Snare	E3	64	Low Conga	E5
41	Low Floor Tom	F3	65	High Timbale	F5
42	Closed Hi-Hat	F#3	66	Low Timbale	F#5
43	High Floor Tom	G3	67	High Agogo	G5
44	Pedal Hi-Hat	G#3	68	Low Agogo	G#5
45	Low Tom	A3	69	Cabasa	A5
46	Open Hi-Hat	A#3	70	Maracas	A#5
47	Low-Mid Tom	B3	71	Short Whistle	B5
48	Hi-Mid Tom	C4	72	Long Whistle	C6
49	Crash Cymbal 1	C#4	73	Short Guiro	C#6
50	High Tom	D4	74	Long Guiro	D6
51	Ride Cymbal 1	D#4	75	Claves	D#6
52	Chinese Cymbal	E4	76	Hi Wood Block	E6
53	Ride Bell	F4	77	Low Wood Block	F6
54	Tambourine	F#4	78	Mute Cuica	F#6

55	Splash Cymbal	G4	79	Open Cuica	G6
56	Cowbell	G#4	80	Mute Triangle	G#6
57	Crash Symbol 2	A4	81	Open Triangle	A6
58	Vibraslap	A#4	82	Shaker	A#6

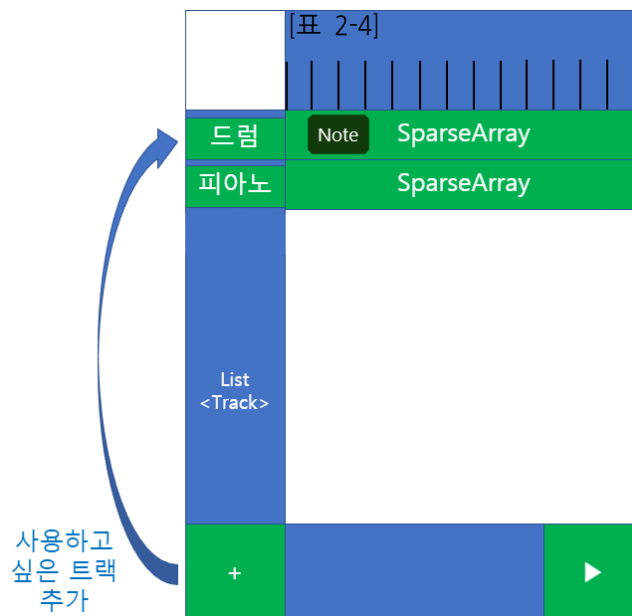
것입니다. 다음은 드럼 채널인 10채널과 General MIDI를 사용하는 나머지 채널에서 쓰이는 음을 정리한 표입니다.

음 값의 뒤에는 속도 값이 붙게 되는데, 속도라는 것은 이벤트에서 해당 음을 누르는 세기(0~127)를 의미합니다. 따라서 음 켜기 이벤트에서 속도를 0으로 주어 음 끄기 이벤트 대신 사용할 수도 있습니다.

SysEx이벤트는 MIDI를 이용해 신디사이저에 메시지를 보내서 신디사이저 설정을 변경하기 위한 것이므로 여기에서는 사용하지 않습니다. 따라서 이에 대한 설명도 생략하겠습니다.

2. MIDI Sequencer UI

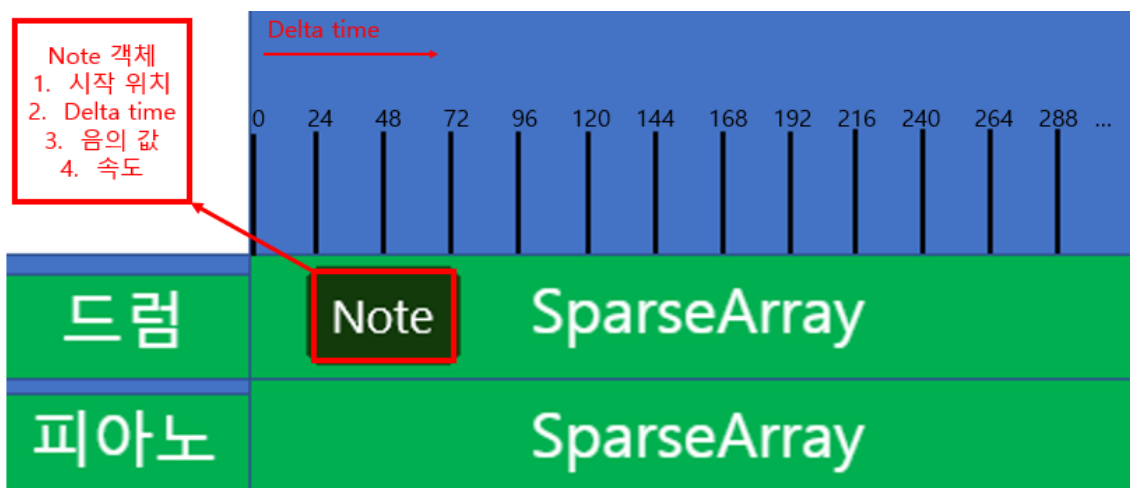
[그림 2-1]



사용자는 트랙을 추가하는 버튼을 누른 후, 어떤 악기의 트랙을 추가할 지 선택하는 Activity에서 악기를 선택하면 결과 값[General MIDI(0-127) 또는 Drum(-1)]을 받아와서 해당 악기의 Track을 생성하여 Track형의 객체 리스트에 추가합니다. 객체 Track은 선택한 악기를 저장하는 int형 변수와 연주데이터를 저장하는 SparseArray로 구성됩니다. Android에서 전용 문법으로 제공하는 SparseArray는 키(key)와 값(value)를

묵어서 하나의 데이터 (entry)로 저장하는 해시맵(HashMap)과 비슷하지만, HashMap은 내부 해시 값에 따라 키순서가 정해지기 때문에 순서를 보장하지 않는 반면에, SparseArray는 키값을 오름차순으로 정렬해서 저장할 수 있고 배열같이 인덱스로 해당 entry를 받아올 수도 있기 때문에 시간에 따라 드문드문 있는 연주 데이터를 저장하고 순서대로 받아와서 MIDI파일로 만들기에는 최적의 데이터 구조라고 생각했습니다.

[그림 2-2]



SparseArray에 연주데이터 객체 Note가 저장되는 방식은 [그림 2-2]와 같습니다. 한 칸의 기준을 4분음표(24 delta time)로 잡았을 때, 4분음표만큼 쉰 후 2분 음표만큼 연주시키려한다면 그림과 같이 한 칸을 뒀 2-3번째 칸에 해당하는 모든 delta time(24~71)을 키 값으로 SparseArray에 <키, Note>쌍을 추가합니다. 여기에서 Note객체는 시작 위치 24, delta time 48, 사용자에게 의해 설정되는 음의 값, 속도로 구성됩니다. 재생 버튼을 누르게 되면 이러한 방식으로 작성된 여러 개의 트랙 List<Track>을 매개 변수로 넘겨서 앞서 설명한 MIDI File Format에 따라 MIDI 파일(.mid)을 바이트로 작성한 후 Android에서 멀티미디어 지원을 위해 제공하는 미디어 API인 android.media 패키지의 MediaPlayer를 활용하여 재생시키게 됩니다. 여기에 보컬 녹음 기능을 더하려면 MediaPlayer가 재생되는 동안 android.media 패키지의 AudioRecord를 사용하는 별개의 스레드를 생성하여 녹음을 시켜야 합니다. AudioRecord를 통해 녹음을 하게 되면 오디오 데이터를 0~65535 범위로 샘플링시킨 PCM(Pulse Code Modulation) 데이터가 생성되게 되는데, PCM은 Raw데이터이므로 재생이 지원되는 형식으로 변환을 해서 사용해야 합니다. 따라서, PCM데이터를 Waveform Audio File Format(.wav)으로 가공해야 했습니다.

3. WAV File Format

WAV파일은 마이크로소프트 사에서 만든 RIFF(Resource Interchange File Format) 컨테이너 형식으로 구성되며, 세 가지의 Chunk(RIFF Chunk, FMT Chunk, DATA

Chunk)로 이루어 집니다.

가. RIFF Chunk

RIFF Chunk는 RIFF 형식임을 알리는 아스키 코드 문자열(Chunk ID), 전체 파일 크기에서 Chunk ID와 자기 자신인 Chunk Size를 제외한 값(Chunk Size), 파일의 포맷을 알리는 아스키 코드 문자열(Format)로 구성됩니다.

1) RIFF 형식임을 알리는 아스키 코드 문자열(4바이트)

Chunk ID “RIFF”를 아스키 코드로 표현하며 각각 R(82), I(73), F(70), F(70) 입니다. 이와 같이 ASCII를 고유 식별자로 사용하는 데이터 형식을 FourCC라고 부릅니다.

2) 전체 파일 크기에서 Chunk ID와 자기 자신인 Chunk Size를 제외한 값(4바이트)

전체 파일 크기에서 위의 Chunk ID(4바이트)와 자기 자신의 크기(4바이트)를 제외한 값이므로, 전체 파일 크기 - 8바이트를 4바이트 형식으로 표현한 값이라고 보면 되겠습니다. 그런데 좀 특이한 것은 연속되는 바이트 블록이 있을 경우 보통은 데이터의 최상위 바이트부터 배치시키는 빅 엔디안(Big Endian) 바이트 순서를 사용하지만 여기에서는 데이터의 최하위 바이트부터 배치시키는 리틀 엔디안(Little Endian) 바이트 순서를 사용한다는 것입니다.

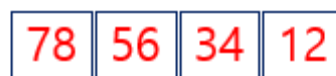
[그림 2-3]

※ 바이트 배열 12 34 56 78

Big Endian 방식



Little Endian 방식



3) 파일의 포맷을 알리는 아스키 코드 문자열(4바이트)

파일의 형식을 나타내며, wave 파일을 작성할 것이기 때문에 이 자리에는 “WAVE”라는 문자가 아스키 코드로 들어가야 합니다. 따라서 각각 W(87), A(65), V(86), E(69)입니다.

나. FMT Chunk

FMT Chunk는 Format Chunk Marker(Chunk ID), FMT Chunk에서 Chunk Size 뒤에 이어질 값의 크기(Chunk Size), Audio Format, Number Of Channel, Sample Rate, Byte Rate, Block Align, Bit Per Sample로 구성됩니다.

1) Format Chunk Marker 아스키 코드 문자열(4바이트)

Chunk ID “fmt ”를 아스키 코드로 표현하며 각각 f(102), m(109), t(116), 공백(0)입니다.

2) FMT Chunk에서 Chunk Size 뒤에 이어질 값의 크기(4바이트)

Audio Format(2바이트), Number Of Channel(2바이트), Sample Rate(4바이트), Byte Rate(4바이트), Block Align(2바이트), Bit Per Sample(2바이트)로 구성되기 때문에 16바이트입니다. 리틀 엔디안 바이트 순서를 사용하므로 16, 0, 0, 0이 될 것입니다.

3) Audio Format(2바이트)

오디오의 형식을 지정하며, PCM이므로 1이 들어갑니다. 리틀 엔디안 바이트 순서를 사용하므로 1, 0입니다.

4) Number Of Channel(2바이트)

오디오의 채널 수를 지정합니다. 단일 채널(MONO)을 사용할 것이므로 1이 들어갑니다. 리틀 엔디안 바이트 순서를 사용하므로 1, 0입니다.

5) Sample Rate(4바이트)

샘플 레이트는 1초 동안의 소리를 몇개의 조각으로 쪼개서 저장하는가를 말합니다. 가장 많이 사용되는 44100 샘플 레이트를 사용하였고, 리틀 엔디안 바이트 순서를 사용하므로 68, 172, 0, 0입니다.

6) Byte Rate(4바이트)

Sample 1개가 차지하는 Byte 수(2) * Sample Rate(44100) * 채널 수(1) 의 값을 구하면 되기 때문에 88200이며, 리틀 엔디안 바이트 순서를 사용하므로 136, 88, 1, 0입니다.

7) Block Align(2바이트)

Sample 1개가 차지하는 Byte 수(2) * 채널 수(1)의 값을 구하면 되기 때문에 2이며, 리틀 엔디안 바이트 순서를 사용하므로 2, 0입니다.

8) Bit Per Sample(2바이트)

Sample 1개를 몇 bit로 나타낼 것인지를 나타냅니다. 16bit로 나타낼 것이며 리틀 엔디안 바이트 순서를 사용하므로 16, 0입니다.

다. DATA Chunk

DATA Chunk는 DATA Chunk임을 알리는 Chunk ID, PCM 데이터의 Size(Chunk Size), PCM 데이터로 구성됩니다.

1) DATA Chunk ID 아스키 코드 문자열(4바이트)

Chunk ID “data”를 아스키 코드로 표현하며 각각 d(100), a(97), t(116), a(97)입니다.

2) PCM 데이터의 Size(4바이트)

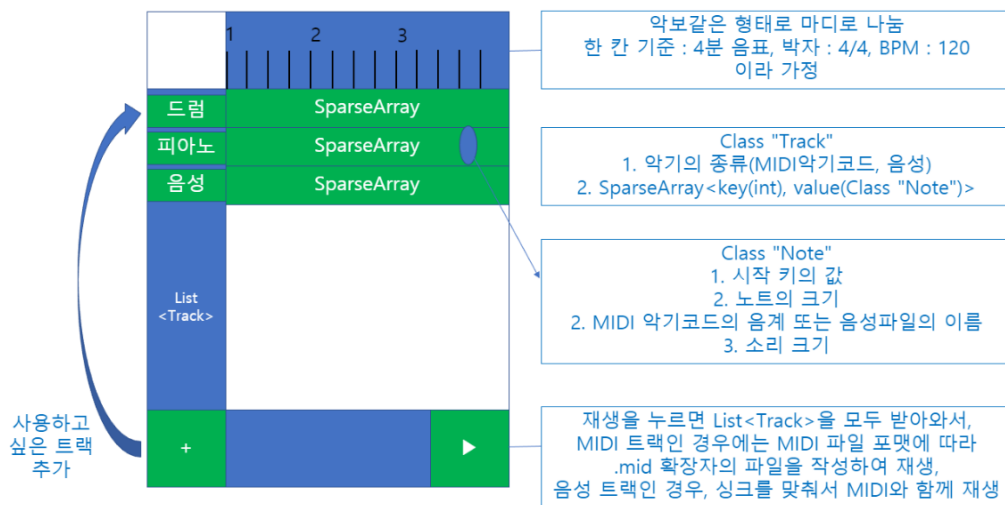
이 뒤에 올 PCM 데이터의 길이를 리틀 엔디안 바이트 순서로 적습니다.

3) PCM 데이터

PCM 데이터를 리틀 엔디안 바이트 순서로 적습니다.

Ⅲ. 작품 구현 및 실험결과

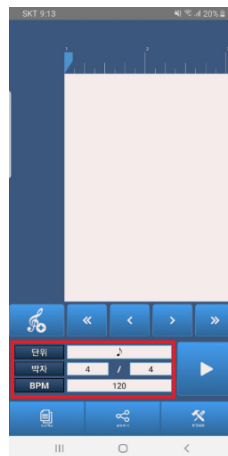
[그림 3-1]



원래의 계획은 [그림 3-1]처럼 트랙에 보컬까지 넣어서 원하는 구간에 보컬을 녹음하도록 하는 것이었으나, 이 방식대로 한다면 보컬은 MIDI가 아니기 때문에 List<Track>에서 보컬과 MIDI를 분리하여 따로 동작시킨 후 싱크를 조절해주어야 하는 번거로운 작업을 거쳐야 하며, 보컬 트랙의 Note객체 하나당 오디오 파일이 하나씩 저장되어야 하기 때문에 여러 프로젝트의 오디오 파일을 저장하게 된다면 디바이스의 저장 공간을 너무 많이 차지하게 되어버릴 수가 있습니다. 따라서, 사용자가 자신의 음악을 다른 사람들과 공유하려 할 때, 일회성으로 녹음하도록 했습니다. 안드로이드 프레임워크를 통해 애플리케이션을 구현하였으며, 애플리케이션의 동작은 다음과 같습니다.

1. 사용자는 한 칸의 기준이 되는 음표, 음악의 박자와 빠르기를 설정합니다.

[그림 3-2]



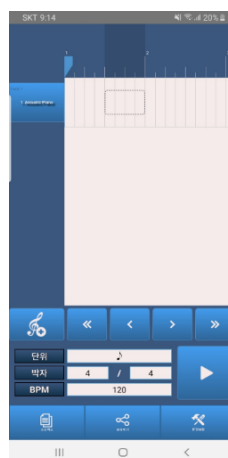
2. 트랙 추가 버튼을 눌러서 사용할 악기의 트랙을 추가합니다.

[그림 3-3]



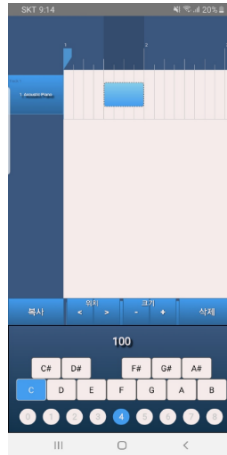
3. 뷰의 터치 이벤트를 통해 SparseArray의 각 키에 Note를 추가합니다.

[그림 3-4]



4. 값을 설정하는 창을 띄워 각 키들이 참조하고 있는 Note의 값을 수정합니다.

[그림 3-5]



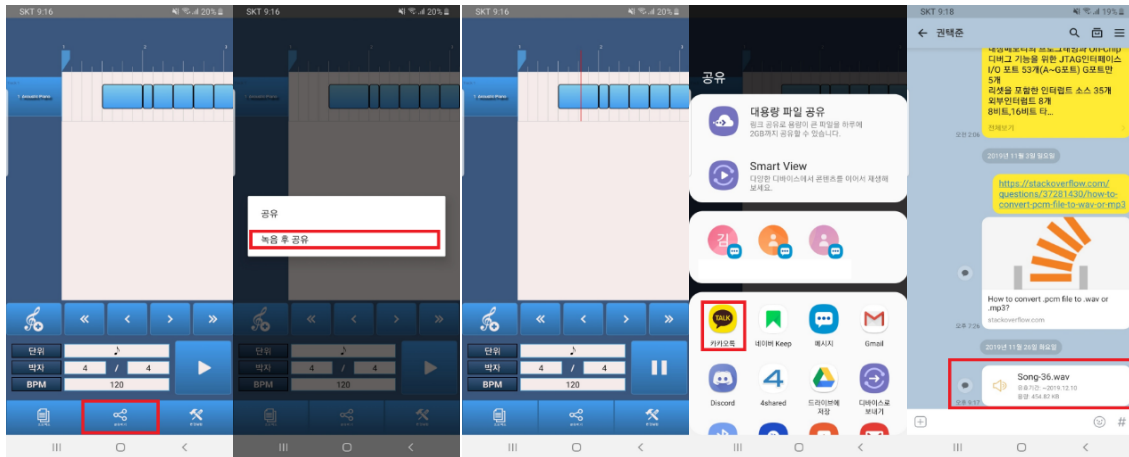
5. 2-4의 과정을 반복하여 작곡을 마친 후, 재생 버튼을 누르면 작성한 정보를 토대로 MIDI 파일을 작성하여 MediaPlayer로 재생시킵니다.

[그림 3-6]



6. 하단의 [공유하기] 버튼을 누르고 [녹음 후 공유]를 선택하면 5의 재생 스투드와 AudioRecord를 이용한 보컬녹음 스투드를 같이 실행시켜 녹음을 하고 WAV 파일을 생성하여 공유합니다.

[그림 3-7]



IV. 결론

애플리케이션의 제작을 통해 누구나 쉽게 작곡할 수 있는 MIDI의 장점을 살리면서 재생되는 MIDI에 사용자의 보컬을 녹음하는 기능까지 추가되어 한정된 소리만을 표현할 수 있었던 기존의 한계점을 보완할 수 있었습니다. 편의성을 개선하고 오류를 수정한 후 애플리케이션 마켓에 등록할 예정입니다.

V. 참고 문헌

- [1] 안드로이드 앱 프로그래밍 - 정재곤 지음, 이지스 퍼블리싱 출판
- [2] WAV 파일 헤더 구조 <https://crystalcube.co.kr/123>
- [3] Big Endian과 Little Endian
https://chortle.ccsu.edu/AssemblyTutorial/Chapter-15/ass15_3.html
- [4] 아스키 코드 표 참고
<https://m.blog.naver.com/PostView.nhn?blogId=kimkwon429&logNo=220723064434&proxyReferer=https%3A%2F%2Fwww.google.com%2F>
- [5] MIDI 파일의 구조
<https://www.csie.ntu.edu.tw/~r92092/ref/midi/>
- [6] General MIDI Instrument and drum kit lists
https://www.pgmusic.com/tutorial_gm.htm